

JAVASCRIPT AVANZADO

CONTENIDO:

1. Fundamentos POO
2. Objetos
3. Clases
4. Constructores
5. Herencia
6. Puntero this
7. Mapas
8. Sets
9. Programación funcional
10. Operaciones sobre objetos
11. Programación asíncrona

1. Fundamentos POO

Características POO

■ Existen cuatro conceptos fundamentales dentro de la Programación Orientada a Objetos que se relacionan entre sí y que nos permitirán tener las riendas de nuestro código:

➡ **Abstracción:** proceso mental de extracción de las características esenciales de algo, ignorando los detalles superfluos.

➡ **Encapsulación:** proceso por el que se ocultan los detalles del soporte de las características esenciales de una abstracción.

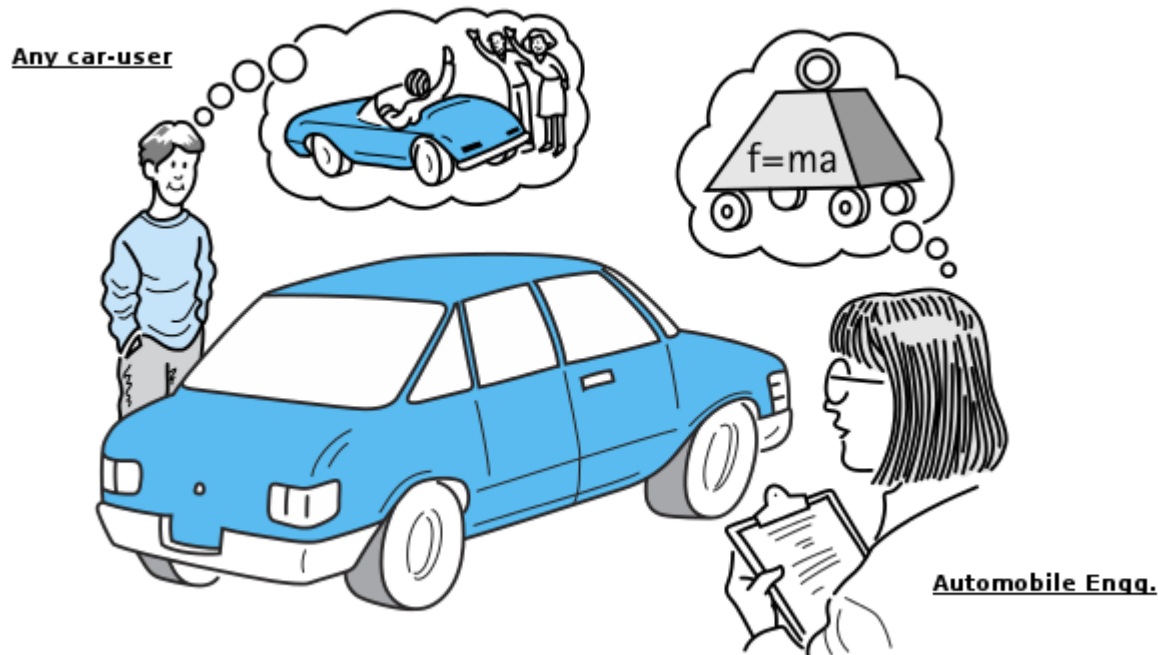
➡ **Modularización:** proceso de descomposición de un sistema en un conjunto de módulos o piezas independientes y cohesivos (con significado propio). Lo adecuado es conseguir los mínimos acoplamientos.

➡ **Herencia:** proceso de estructuración por el que se produce una organización (jerarquía) de un conjunto de elementos en grados o niveles de responsabilidad, incumbencia o composición entre otros.

➡ **Polimorfismo:** proceso para ver el objeto de diferentes modos

Abstracción

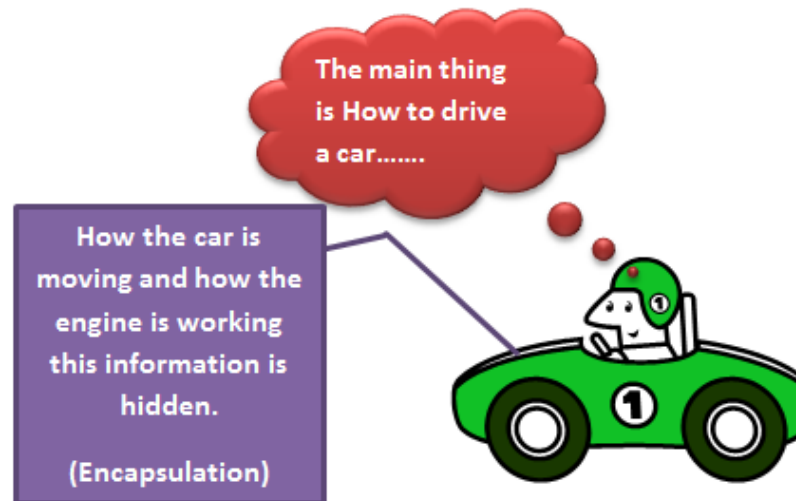
Consiste en **aislar** por supresión intencionada, u ocultamiento, **las cualidades de un objeto** para considerarlas aisladamente o para considerar el mismo objeto en su pura esencia o noción.



An abstraction includes the essential details relative to the perspective of the viewer

Encapsulamiento

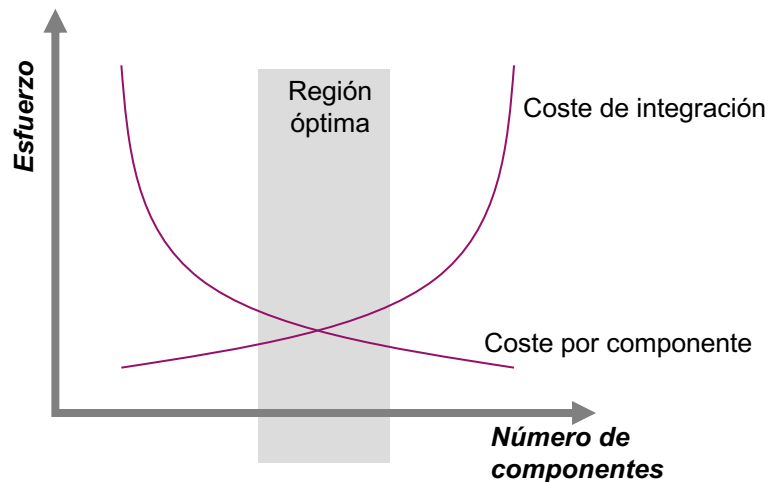
- Significa reunir a todos los elementos que pueden considerarse pertenecientes a una misma entidad, al mismo nivel de abstracción. Esto permite aumentar la cohesión de componentes del sistema.
- Técnica para la ocultación de información que ayuda a definir mejor los módulos.
- Los módulos deben diseñarse para que la información que manejan está oculta a los otros módulos, de tal forma que entre módulos intercambien el volumen mínimo posible de información.



Modularidad

- Consiste en dividir el software en diferentes componentes, resolver cada uno por separado y por último unirlos en una solución única.
- Ante un problema complejo es más fácil resolverlo cuando se divide en piezas manejables.

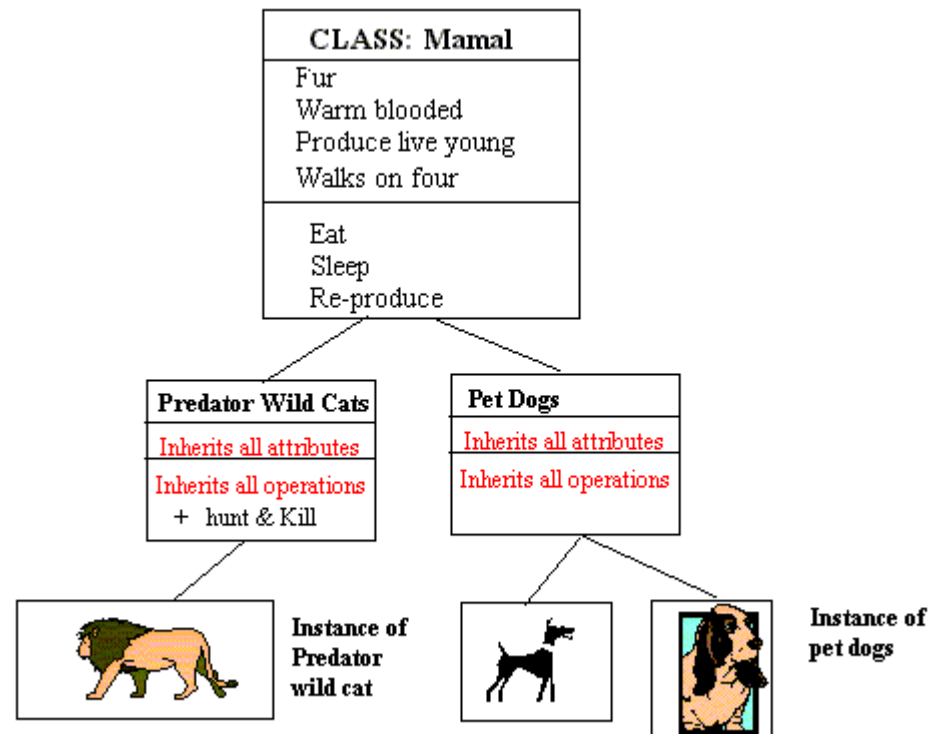
¿Cuál es el número óptimo de componentes en el que debo dividir un programa?



- Existe una región óptima en la que tanto el esfuerzo de integración como la dificultad de cada componente es baja.

Herencia

- Las clases no están aisladas, sino que se relacionan entre sí, formando una jerarquía de clasificación.
- Los objetos heredan las propiedades, relaciones y los comportamientos de todas las clases a las que pertenecen.
- La herencia organiza y facilita el polimorfismo y el encapsulamiento, permitiendo compartir y extender las propiedades y el comportamiento sin tener que volver a implementarlo.



Polimorfismo

■ Permite **referirse a objetos de clases diferentes mediante el mismo elemento de programa** y realizar la misma operación de diferentes formas, según sea el objeto que se referencia en ese momento.



2. Objetos

Objetos

- Un **objeto** es un recurso dinámico que se crea y se almacena en memoria durante un tiempo determinado. A los objetos también se les conoce con el nombre de **instancias** de clase porque realmente esto es lo que son, una copia de la clase con unos valores determinados.

```
var alumno1 = new Alumno("Juan", "Lopez", 7.5);  
var alumno2 = new Alumno("Maria", "Arias", 5.6);
```

3. Clases

Clases

- Una **clase** la podríamos definir como la plantilla a partir de la cual se va a generar el objeto.

```
class Alumno{
```

```
}
```

Propiedades

- Las propiedades o también llamadas atributos o campos recogen las características de la clase.
- Una propiedad se entiende como una variable global a la cual se puede acceder desde cualquier otro recurso dentro de la misma clase y dependiendo del modo de acceso, también se podrá acceder a ella desde otras clases.

```
class Alumno{  
    nombre;  
    apellido;  
    nota;
```

Métodos

- Utilizamos los métodos para introducir el código de nuestra aplicación. Podremos invocar a un método tantas veces como se quiera por lo cual nos permite una reutilización de código para aquellas tareas repetitivas.
- Igual que ocurre con las propiedades, un método se puede invocar desde otro recurso declarado en la misma clase y dependiendo de su acceso, se podrá llamar desde otras clases.

```
mostrarInformacion(){  
    return "NOMBRE :: " + this.nombre +  
        " | APELLIDO :: " + this.apellido +  
        " | NOTA :: " + this.nota;  
}
```

Clases encapsuladas

- Una clase encapsulada es una clase que declara todas sus propiedades como privadas y solo se puede acceder a ella a través de los métodos get() y set().

```
// Propiedades privadas  
#nombre;  
#apellido;  
#nota;
```

Clases encapsuladas

- Toda propiedad privada va a tener asociados dos métodos públicos (get y set) a través de los cuales vamos a poder modificar el valor de la propiedad (set) o recuperar su valor (get).

```
getNota(){  
    return this.#nota;  
}  
  
setNota(nuevaNota){  
    this.#nota = nuevaNota;  
}
```

Clases idempotentes

■ Si queremos que solo se lean las propiedades o atributos de la clase y que no se puedan modificar, entonces tendremos los métodos `get()` pero no los métodos `set()`.

```
class Alumno{
    #nombre;
    #apellido;
    #nota;

    constructor(nombre, apellido, nota){
        this.#nombre = nombre;
        this.#apellido = apellido;
        this.#nota = nota;
    }

    getNombre(){
        return this.#nombre;
    }

    getApellido(){
        return this.#apellido;
    }

    getNota(){
        return this.#nota;
    }
}
```

4. Constructores

Constructores

- Un constructor es como una especie de método que se invoca a la hora de crear un objeto.
- Puede ser que reciba argumentos o no. Estos no son obligatorios al igual que los métodos.

```
// Esto se ejecuta cada vez que creamos un nuevo objeto (new)
constructor(nombre, apellido, nota){

    // Los datos que hemos recibido en el constructor
    // los asignamos a las propiedades del objeto
    this.nombre = nombre;
    this.apellido = apellido;
    this.nota = nota;
}
```

5. Herencia

Herencia

Una de las características de la programación orientada a objetos es la herencia. El termino herencia es conocido por todos, quien no ha oído hablar de una herencia en la familia. Nosotros no vamos a heredar dinero ni propiedades pero si vamos a aprender a heredar recursos de otras clases.

```
class Figura{  
    constructor(x, y){  
        this.x = x;  
        this.y = y;  
    }  
  
    area(){  
        return 0;  
    }  
  
    posicion(){  
        // [x,y]  
        return "[" + this.x + "," + this.y + "];"  
    }  
  
    mostrarDatos(){  
        return "x: " + this.x + " y: " + this.y;  
    }  
}  
  
class Circulo extends Figura{  
    constructor(x, y, radio){  
        // Estamos pasando x e y al constructor de la superclase  
        super(x,y);  
        this.radio = radio;  
    }  
}
```

Sobreescritura de métodos

- Además de poder agregar atributos para generar clases nuevas a partir de una clase existente, es posible modificar el comportamiento de la clase heredada.
- Si se define un método en una subclase de tal forma que su nombre y la lista de argumentos son idénticos a los de la superclase (la clase de nivel superior), se dice que el nuevo método sobrescribe al anterior.
- En la imagen vemos como en la clase Circulo se sobrescribe el método area.

```
class Circulo extends Figura{  
    > constructor(x, y, radio){ ...  
        }  
  
    area(){  
        return Math.PI * Math.pow(this.radio, 2);  
    }  
}
```

Llamar a métodos sobreescritos

- Un método de una subclase puede llamar a un método de una superclase utilizando el puntero super.
- Se usa para hacer referencia a las variables miembro o a los métodos de la superclase.
- En la imagen vemos como en el método mostrarDatos se invoca al método sobreescrito de la superclase, de esta forma nos evitamos tener que mostrar los valores de las coordenadas x e y, ya que se hará desde la superclase.

```
class Circulo extends Figura{  
    constructor(x, y, radio){ ...  
    }  
  
    area(){ ...  
    }  
  
    mostrarDatos(){  
        return super.mostrarDatos() + " radio: " + this.radio;  
    }  
}
```

6. Puntero this

Puntero this

- El puntero this es la forma de acceder a los recursos de la clase (propiedades y métodos).
- Se le llama puntero porque apunta a la propia instancia
- Sirve para resolver ambigüedades entre las propiedades y variables locales.

```
// Esto se ejecuta cada vez que creamos un nuevo objeto (new)
constructor(nombre, apellido, nota){

    // Los datos que hemos recibido en el constructor
    // los asignamos a las propiedades del objeto
    this.nombre = nombre;
    this.apellido = apellido;
    this.nota = nota;
}
```

7. Mapas

Mapas

- Los elementos de un mapa se forman como clave-valor.
- Además tienen las siguientes restricciones:
 - ↳ Las claves duplicadas no están permitidas.
 - ↳ Una clave solo puede referenciar un valor, no varios.

```
// Crear el mapa
var mapa = new Map();

// Agregar elementos al mapa
mapa.set("Pepe", 5.0);
mapa.set("Juan", 7.2);
mapa.set("Maria", 8.2);
mapa.set("Laura", 6.8);
mapa.set("Ismael", 9.3);
mapa.set("Sofia", 8.2);
```

Métodos clase Map

■ La clase Map define una serie de métodos para manipular el mapa, los más interesantes son:

➞ **entrySet**; Devuelve una colección de tipo Set con todos los elementos (clave-valor).

➞ **keys**; Devuelve una colección de tipo Set con todas las claves del mapa.

➞ **values**; Devuelve un objeto con todos los valores del mapa.

```
// Solo quiero mostrar las claves
for(var clave of mapa.keys()){
    console.log(clave);
}

// Solo quiero mostrar los valores
for(var valor of mapa.values()){
    console.log(valor);
}

// Mostrar los elementos por pares (key=value)
for(var [clave,valor] of mapa.entries()){
    console.log(clave + " = " + valor);
}
```

Otros métodos

- La clase Map tiene otros métodos. Entre ellos:
 - ➞ **get**; Devuelve el valor de la clave pasada como parámetro.
 - ➞ **delete**; Elimina un elemento pasando su clave.
 - ➞ **size**; Devuelve el tamaño del mapa. El numero de elementos que contiene.

```
// Recuperar un elemento  
console.log("Nota de Laura: " + mapa.get("Laura"));
```

```
// Eliminar un elemento  
mapa.delete("Juan");
```

```
// Mostrar numero de elementos  
console.log("Numero de elementos: " + mapa.size);
```

8. Sets

Set

- Es un tipo de colección donde no permite elementos duplicados.
- Si intentamos introducir un elemento repetido simplemente lo ignora sin generar una excepción o error de compilación.

```
// Crear un Set
var set = new Set();

// Agregar valores a un set
set.add("Juan");
set.add("Laura");
set.add("Ismael");
set.add("Sofia");
set.add("Pepe");
set.add("Antonio");
```

Métodos de la clase Set

■ Además del método `add` que utilizamos para agregar elementos a la colección de tipo `Set`, tenemos los siguientes:

➞ **`has`**: Devuelve un valor booleano indicando si existe ese elemento en la colección o no.

➞ **`delete`**: Elimina un elemento.

➞ **`size`**: Devuelve el numero de elementos que contiene la colección.

```
// Comprobar si existe un valor
console.log("Tenemos a Laura? " + set.has("Laura"));
```

```
// Borrar un elemento
set.delete("Laura");
```

```
// Ver numero de elementos
console.log("Numero de elementos: " + set.size);
```

9. Programación funcional

Definición

- El nombre ya lo sugiere: la programación funcional o functional programming se centra en las **funciones**.
- En un programa funcional, todos los elementos pueden entenderse como funciones y el código puede ejecutarse mediante **llamadas de función secuenciales**.

```
var nombres = ["Pepe", "Juan", "Antonio", "Laura", "Sofia", "Ismael"];

for(var i=0; i<nombres.length; i++){
|   console.log(nombres[i]);
}

// Aplicando programacion funcional
nombres.forEach(nombre => {
|   console.log(nombre);
});
```

Principio de inmutabilidad

- Todo proceso que se lleva a cabo dentro de la función no modifica los valores iniciales, ya que se hace sobre una copia.

```
// El array no cambia, solo se aplican las modificaciones en una copia
// se le llama principio de inmutabilidad.
numeros.map(num => num * 2);
console.log(numeros);
```

Funciones principales

- **map:** El resultado conserva la forma del argumento de entrada, probablemente con diferente tipo.
- **filter:** El resultado tendrá la misma forma que el argumento de entrada, probablemente de menor longitud.
- **reduce:** El resultado puede quedar totalmente transformado.

```
numeros = numeros.map(num => num * 2);  
console.log(numeros);
```

```
// Filtrar elementos de una coleccion  
var mayores = numeros.filter(num => num > 5);  
console.log(mayores);
```

```
var resultado = [5,9,2,6].reduce((total, num) => total + num);  
console.log(resultado);
```

Combinar funciones

Podemos encadenar funciones:

```
var resultado3 = ["5","9","2","6"]  
  .map(dato => parseInt(dato))  
  .filter(num => num > 5)  
  .reduce((total, num) => total + num);
```

En este ejemplo primero se convierte cada elemento de tipo texto a un numero entero, luego se filtran los números mayores que 5 y por ultimo se calcula su suma. Mostrando por ultimo su valor por consola:

```
console.log(resultado3);
```

10. Operaciones sobre objetos

Operaciones sobre objetos

- Accedes a las propiedades de un objeto con una simple notación de puntos:

```
objectName.propertyName
```

- puedes llamar al método en el contexto del objeto de la siguiente manera:

```
object.methodname(params);
```

11. Programación asíncrona

Síncrono vs Asíncrono

- Se refiere a cuando tendrá lugar la respuesta:

➡ **Síncrono:** es frecuente emplear 'bloqueante' y 'síncrono' como sinónimos, dando a entender que toda la operación de entrada/salida se ejecuta de forma secuencial y, por tanto, debemos esperar a que se complete para procesar el resultado.

➡ **Asíncrono:** la finalización de la operación *I/O* se señala más tarde, mediante un mecanismo específico como por ejemplo un *callback*, lo que hace posible que la respuesta sea procesada en diferido. Como se puede adivinar, su comportamiento es no bloqueante ya que la llamada *I/O* devuelve inmediatamente.

Callbacks

- Los *callbacks* son la pieza clave para que Javascript pueda funcionar de forma asíncrona. De hecho, el resto de patrones asíncronos en Javascript está basado en *callbacks* de un modo u otro.
- Un *callback* no es más que **una función que se pasa como argumento de otra función**, y que será invocada para completar algún tipo de acción.
- En nuestro contexto asíncrono, un *callback* representa el '*¿Qué quieres hacer una vez que tu operación asíncrona termine?*'. Por tanto, es el trozo de código que será ejecutado una vez que una operación asíncrona notifique que ha terminado.

```
var miCallback = () => {alert("Adios")};  
setTimeout(miCallback, 1000);
```

Promesas

- Una promesa es un objeto que representa **el resultado de una operación asíncrona**. Este resultado podría estar disponible **ahora** o en el **futuro**. Las promesas se basan en *callbacks*.
- Cuando llamamos a una función asíncrona implementada con este patrón, nos devolverá inmediatamente una promesa como garantía de que la operación asíncrona finalizará en algún momento, ya sea con éxito o con fallo.
- Una vez que tengamos el objeto promesa en nuestro poder, registramos un par de *callbacks*: uno para indicarle a la promesa *'que debe hacer en caso de que todo vaya bien'* (resolución de la promesa o *resolve*) y otro para determinar *'que hacer en caso de fallo'* (rechazo de la promesa o *reject*).

```
var url = "https://pokeapi.co/api/v2/pokemon/pikachu";  
var promesa = fetch(url);  
promesa.then(resultado => console.log(resultado.json()));  
promesa.catch(error => console.log(error));
```

Async / Await

- Las palabras clave ***async*** y ***await*** surgieron para simplificar el manejo de las promesas. Son para hacer las promesas más amigables, escribir código más sencillo, reducir el anidamiento y mejorar la trazabilidad al depurar. Pero recuerda, *async* \ *await* y las promesas son lo mismo en el fondo.
- La etiqueta ***async*** declara una función como asíncrona e indica que una promesa será automáticamente devuelta. Podemos declarar como *async* tanto funciones con nombre, anónimas, o funciones flecha.
- Por otro lado, ***await*** debe ser usado siempre dentro de una función declarada como *async* y esperará automáticamente (de forma asíncrona y no bloqueante) a que una promesa se resuelva.

Async / Await

```
var url = "https://pokeapi.co/api/v2/pokemon/pikachu";
async(url) => {
  try{
    var respuesta = await fetch(url);
  } catch (error){
    console.log(error);
  }
}
```